

Qaul (Freifunk) qaul BLE module for Linux.

GSoc 2024

Personal Info

Name: Sukhman Singh

Email: sukhmansinghsaluja@gmail.com

Telephone: +91-9415860440

Time-Zone: India (UTC+5.30)

Github: [sukhman-sukh](#)

University Info

University: Indian Institute of Technology, Roorkee

Major: Computer Science and Engineering

Current Year: 2nd year (2026 expected graduation)

Degree: Bachelor of Technology (B.Tech)

About Me

I am a 2nd Year Bachelor of Technology student at the Indian Institute of Technology, Roorkee. I've been involved in software development for the past 2 years, with several projects deployed to production. I am a member of SDSLabs, a student-run technical club in our institute, and have been involved in software development in various spheres, including cloud and infra development and information security.

I've been passionate about programming and software development since school and have worked on several personal and group projects since coming to college. In my freshman year, I was introduced to website development and deployment including technologies like containerization, orchestration, computer networking, and system architecture in general.

Project

Abstract

qaul is an internet-independent wireless mesh p2p communication application. With qaul, you can communicate directly from one user to another sharing texts, images, audio, videos, and other documents without the concern of privacy breaches ensuring secure, end-to-end communication.

The project aims to create a qaul Ble module in Rust for Linux which is compatible with modules for Android and iOS. The module should provide the following functionalities :

- **BLE advertisements** — The project should be able to listen to and send BLE advertisements to nearby nodes at regular intervals.
- **Establish data communication** — The connection should be able to exchange messages between qaul nodes.

Plan Of Action

- **Initiate BLE GATT server :**
 - Initialize the Bluetooth session and activate the Bluetooth adapter.
 - Initiate local Ble GATT server using the bluer crate and set up Bluetooth GATT Services and Characteristics, which become available for remote servers.
- **Establish communication between devices :**
 - Initiate advertisement to look for nearby qaul nodes with specified service uuid and characteristic uuid.
 - Connect to discovered devices and send rpc messages to libqaul.
 - Write data into the **characteristic.value** if **service.uuid()** and **characteristic.uuid()** are compatible with Android and iOS version.
 - Create a message listener and forward the characteristic value passed in the event to libqaul.

Implementation

This project aims to establish a BLE connection between Linux and other compatible devices.

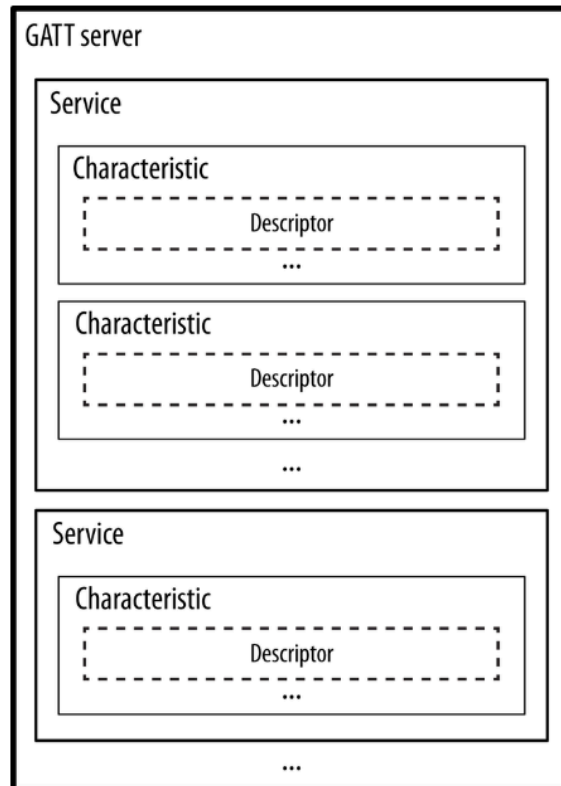
BLE GATT Service —

GATT Client —

A GATT client corresponds to an ATT client. It sends requests to the server and receives responses. It does not know anything in advance about the server, so it must first inquire presence and nature of attributes (Services, Characteristics, and descriptors). After the discovery of service attributes of the server, it can start reading/writing to attributes found on the server based on permissions provided by the server.

GATT Server —

A GATT server corresponds to an ATT server. It receives requests from clients and sends responses back.



GATT data hierarchy.

Services:-

GATT services are a set of similar attributes in one common section of the GATT server. A service contains characteristics or secondary services. For gaul, we are going to create services

```
pub struct Service {
  pub uuid: Uuid,
  pub handle: Option<NonZeroU16>,
  pub primary: bool,
  pub characteristics: Vec<Characteristic>,
  pub control_handle: ServiceControlHandle,
  /* private fields */
}
```

BlueR struct for GATT Service.

For Qaul GATT Server implementation, we have two services — **mainService**, **msgService** and their UUIDs implemented in Android and iOS ble implementation.

Characteristics —

Characteristics can be considered as containers for user data. It has mainly two attributes: descriptors(metadata contained in the characteristic declaration.) and values(contains the user data).

```
pub struct Characteristic {
    pub uuid: Uuid,
    pub handle: Option<NonZeroU16>,
    pub broadcast: bool,
    pub writable_auxiliaries: bool,
    pub authorize: bool,
    pub descriptors: Vec<Descriptor>,
    pub read: Option<CharacteristicRead>,
    pub write: Option<CharacteristicWrite>,
    pub notify: Option<CharacteristicNotify>,
    pub control_handle: CharacteristicControlHandle,
    /* private fields */
}
```

For the Qaul Gatt Server, we have one characteristic for each service — **mainChar**, **msgChar**, and their UUIDs implemented in Android and iOS ble implementation.

Initialize Bluetooth session and Bluetooth Adapter —

```
let session = bluer::Session::new().await?;
let adapter = session.default_adapter().await?;
adapter.set_powered(true).await?;

/// Now adapter can be used for
/// let app_handle = adapter.serve_gatt_application(app).await?; // to register a local gatt server
/// adapter.advertise(); // to start advertisement;
/// adapter.discover_devices(); // to discover devices;
```

Implement Service, Characteristic for gatt_server

```

let (main_service_control, main_service_control_handle) = service_control();
let (main_characteristic_control, main_characteristic_control_handle) = characteristic_control();h
let main_characteristic = Characteristic {
  uuid : UUID_characteristic,
  broadcast: false,
  writable_auxiliaries: false,
  read: Some(CharacteristicRead{
    read: true,
    fun: || {log::info!("Read main_characteristic");}
  }),
  control_handle: main_characteristic_control,
  ..Default::default()
}

let main_service = Service {
  uuid: UUID_service,
  primary: true,
  characteristics: vec![main_characteristic],
  control_handle: main_service_control_handle,
  ..Default::default()
}

let app = Application{ services: vec![main_service],
  ..Default::default()}
adapter.serve_gatt_application(app).await;

```

Establish Communication:-

Once `adapter.advertise()` has started, begin scanning for devices using `adapter.discover_device()`. Once a device is found, check for its absence in the blacklist, validate its services/characteristics UUIDs, and confirm its presence by checking `device.rssi`. If a device is found to satisfy the above conditions, check for its presence in `deviceList`. If it is not present, add it to both `deviceList` and `ignoreList` and send an RPC message to libqaul to update the `discovered_devices` list in the UI. If the device is already present, update its `lastFoundTime`.

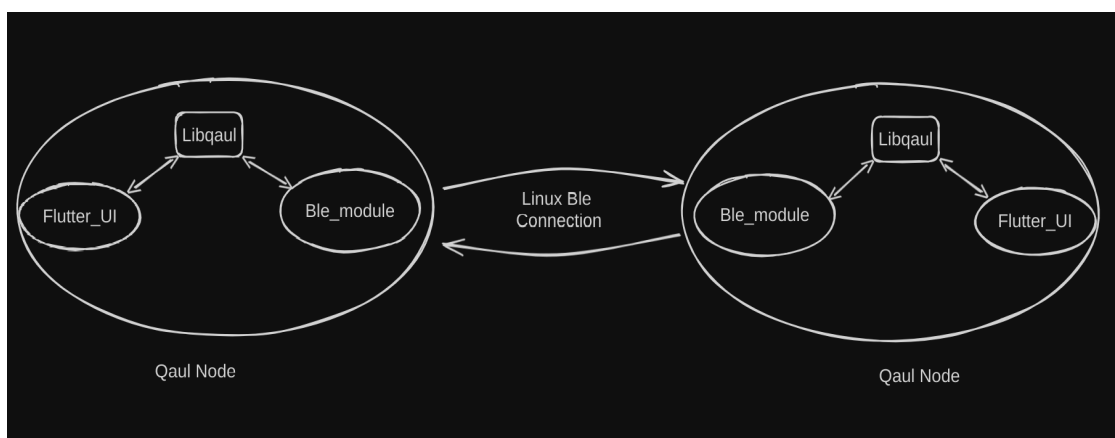
Keep checking for devices out of range by

```
if (bLEDevice.lastFoundTime != null &&
    (bLEDevice.lastFoundTime!! < System.currentTimeMillis() - 5000))
```

Remove from deviceList and ignoreList for all devices out of range.

On receiving **message.DirectSend** request from libqaul, redirect the data to a function that checks for devices in deviceList with specified serviceUUID, characteristicUUID, and qaul-ID of the receiver and write the data to characteristic.value.

To read the characteristic value on updation, we have **bluer::gatt::local::CharacteristicControl** which triggers **bluer::gatt::local::CharacteristicControlEvent** event which gives us an updated value in characteristic which we can send to libqaul to serialize the message to display in UI.



Qaul Bluetooth connectivity architecture

Timeline

GSoC Timeline	
Pre GSoC Period	
April 3 - May 1	• I will be completing my remaining PRs on Qaul and try to complete my assigned issues before the community bonding period. • Resolving new issues Reading more on Qaul's Architecture
Community Bonding Period	
May 1 - May 26	• Get to know more about the Qaul community. • Discussion with my mentor to finalize BLE implementation and architecture.
Coding Period	
May 27 - June 2	• Getting familiar with existing qaul configurations and data flow. • Learn the implementation of various technologies to be implemented during the contribution phase.
June 3 - June 16	• Create a prototype for Linux BLE connection using bluer module. • Create a GATT server and implement its service and characteristic configurations. • Implement GATT server callbacks.
June 17 - June 23	• Create a GATT client for reading and writing data during communication. • Establish device connection using the bluer crate. • Establish communication and ensure message transfer between two Linux devices using BLE.
June 24 - July 1	• Integrate the ble-module in the Qaul codebase. • Establish communication between the Qaul Linux node and the ble-module.
July 2 - July 8	• Buffer time to complete previous tasks(if any)

	• Prepare documentation of work done till now. • Submit mid-term evaluation report.
Mid evaluation	
July 12 - August 2	• Create an internal service to transfer protobuf serialized data from BLE communication to libqaul. • Establish communication between pre-existing Android and iOS qaul nodes. • Ensure compatibility between devices/
August 3 - August 11	• Work on optimization of existing BLE_module for a more stable connection, preserving compatibility with other types of devices.
August 12 - August 18	• Future goals include implementing a prototype for extending advertisement and incorporating M2 Phy or coded Phy BLE. • Buffer time to complete previous tasks(if any)
August 19 - August 25	• Prepare a documentation of work done. • Prepare a final evaluation report.
Final evaluation	

Deliverables :

1. Creating a stable working ble_module for Linux including fixing, and testing the current prototype.
2. Establish communication of the ble_module with libqaul.
3. Make sure the Linux Bluetooth code is interoperable with Android and iOS Bluetooth modules.
4. Try to optimize the Bluetooth module for better stability and connection.

Future Goals:

1. Implementation of a prototype for extended advertisement for topology updates and qaul-ID spreading.
2. Implement a more advanced version of BLE connection — BLE Coded Phy for longer range and less data retransmission (for Bluetooth-5.0 and above) which can be achieved through secondary channels: extended advertisement.

Related Experience :

Pinger —

- Written in Go, It is an open-source implementation of platforms used to create status pages.to manage users, checks, pages, alerts, etc.

Gbemu —

- Written in C++, Gbemu is a Gameboy emulator platform that gives you a nostalgic feeling of a Gameboy device by creating emulation.

Gaspar —

- It is an intelligent Platform as a Service (PaaS) written in Golang used to deploy and manage applications and databases in any cloud topology. It builds and runs applications in docker containers directly from the source code.

Watchdog —

- Watchdog is a personalized server access management tool that keeps track of all the administrative rights attempts (like sudo and su) on the server (via SSH) and allows/disallows log-in attempts based on the public key of the user and logs all activity

Being part of several projects under SDS Labs, I am comfortable with Rust, Dart, Golang, C++, Javascript, Typescript, Solidity, and Verilog. I have sound knowledge of Docker and Kubernetes. Apart from development, I also try my hands on CTFs(capture the flag security events).

Open Source Contributions :

- Pinger: I worked on pinger where I implemented a gRPC connection using protobufs between microservices in Golang.
 - <https://github.com/sdslabs/pinger/pull/110/commits/33ab05233662d3700240e5ac4d057128ec4bc066>
 - <https://github.com/sdslabs/pinger/pull/110>
- Gbemu: I wrote a debugger in a Gameboy emulator which can apply breakpoints, read/write values of registers, analyze sprites and emulator screen pixels, and rendering.
 - <https://github.com/sdslabs/gbemu/pull/25/commits/1f8c7a609e3c0d953777ff6a6efac0af80438022>
 - <https://github.com/sdslabs/gbemu/pull/25/commits/c258d053bd54ba26ea3442971c936b72a6a4c2b1>
 - <https://github.com/sdslabs/gbemu/pull/25/commits/c7e93c2986d379651c9035a0c64542c01b28faac>

- <https://github.com/sdslabs/gbemu/pull/25/commits/6aa03206413637fd143b6d1d23cd02d6024a505e>
- <https://github.com/sdslabs/gbemu/pull/25/commits/581fb2fde8327a9fe3c38e9d4d9ac928217c59de>
- <https://github.com/sdslabs/gbemu/pull/25>
- Watchdog: Dockerized watchdog server and its homepage, configured reverse proxy using nginx, and hosted the on aws server.
 - <https://github.com/sdslabs/watchdog-homepage/pull/8>

Motivation

I have always been excited by the prospect of converting ideas into products with real-world impact and have been captivated by the concept of open source ever since I was first introduced to it. Since then, I have wanted to be a part of this community and have been trying to contribute in the small ways I could by contributing to the open-source community. GSoC provides the perfect opportunity to jump right into the Open Source World with the opportunity to work on awesome and even bigger open-source projects with experienced developers. That's why I am motivated to participate in GSoC, as it will enhance my knowledge and development skills and allow me to work on industry-scale projects under experienced mentors.

I was always inspired by the idea of a world built on free and decentralized architecture. When I read the project's description, I was motivated to be a part of a community that aims to achieve borderless, decentralized communication which is at par with the established centralized application.

My interest peaked when I came across a few initiatives by this community like the TED Talk “Can you hear me?” and Zone*Interdite, thus I wanted to become part of this journey.

Qaul Open-Source Contribution

- Tried to understand the qaul UI architecture and file structure.
- Tried understanding libqaul <-> UI communication through serialized protobufs.
- Raised a PR implementing an audio-chat feature through qaul([#615](#))
- Understood data flow and how BLE connection is established in Android and iOS for qaul.

Post-GSoC

Even after completing my project, I plan on continuing to contribute to Qaul even after GSoC along with solving any other issues or maintenance needs that come up in the future regarding the above-mentioned features. Having picked up many development skills, my primary focus would be to help the project and the community grow.

Availability

I can dedicate more than 30 hours a week. The college's summer vacations are scheduled from mid-May to July. Therefore, there would be no classes for the first two months of the program's coding period and no exams throughout the coding period. My college will reopen on July 18th. During this time, I could work more than 25 hours a week.

. I generally work from 11 AM to 1 AM IST, although I am open to changing/adjusting my working hours if needed.

Other than this project, I have no commitments planned for the summer. I shall keep my status posted to all the community members every week and maintain transparency in the project.

My Matrix ID — @sukhman2:matrix.org

References

- <https://qaul.net/>
- <https://qaul.github.io/qaul.net/#/qaul/README>
- <https://github.com/qaul/qaul.net/issues/548>
- <https://developer.android.com/reference/android/bluetooth/le/package-summary>
- <https://docs.rs/bluer/latest/bluer/>

- https://docs.rs/bluer/latest/src/gatt_server_io/gatt_server_io.rs.html#73
- <https://github.com/qaul/qaul.net/issues/314>
- <https://docs.silabs.com/bluetooth/4.0/general/connections/using-2m-and-le-coded-phy-bt5>